

Hippocrates AI — Spaced Repetition System (SRS)

Complete Technical & Conceptual Documentation

Table of Contents

- 1. Overview & Philosophy
- 2. The Data Model
- 3. Question Types
- 4. Question Generation Pipeline
- 5. The Approval & Scheduling Pipeline
- 6. The Quiz Session — Loading & Selection
- 7. Quota-Based Card Selection Algorithm
- 8. The Frontend Queue Runner — Card Ordering
- 9. Answering a Card — Mastery & Requeue
- 10. The Cooldown (Re-queue) System
- 11. The Mastery Streak & Graduation
- 12. Inter-Session Scheduling — The Next Review Date
- 13. Daily Study Streak System
- 14. Dashboard Metrics
- 15. Streak Reset Logic
- 16. End-to-End Flow Walkthrough
- 17. Key Constants Reference

1. Overview & Philosophy

Hippocrates AI uses a **custom Spaced Repetition System (SRS)** built from the ground up for medical education. It intentionally departs from the classical SM-2 algorithm (used by Anki) in two fundamental ways:

SM-2 / Anki	Hippocrates AI SRS
Rating-driven (1–4 ease factor)	Correctness-driven (correct / incorrect only)
Long, pre-calculated interval schedules	Short intra-session cooldowns + fixed inter-session intervals
Ease factor drifts based on performance	Fixed intervals (3 days → 5 days)

The goal is simple: **you must answer a question correctly 3 consecutive times in a single study session** before it is considered mastered and rescheduled. This forces genuine, not lucky, recall.

2. The Data Model

The entire SRS is built on three database tables.

Question

Stores the raw question content, its type, its status, and a link back to the exact block of text in the material it was generated from.

Field	Description
<code>id</code>	UUID primary key
<code>material</code>	FK → the source material (uploaded document)
<code>source_block</code>	FK → the exact text block the question was generated from
<code>heading_2_block</code>	FK → the nearest H2 heading, used for grouping
<code>question_type</code>	<code>mcq</code> , <code>true_false</code> , <code>fill_in</code> , or <code>applies</code>
<code>status</code>	<code>pending</code> → awaiting review; <code>approved</code> → entered the SRS; <code>rejected</code>
<code>selected_text</code>	The specific text the user highlighted when generating
<code>payload</code>	JSON blob holding the full question content (options, answers, etc.)

QuestionSchedule

One row **per question per user**. This is the SRS state machine — it is the heart of the system.

Field	Description
<code>question</code>	FK → Question
<code>user</code>	FK → User
<code>scheduled_date</code>	The calendar date on which this card is due for review
<code>streak</code>	Consecutive correct answers in the current session (0–3)
<code>review_count</code>	How many times this card has been fully mastered across all sessions
<code>last_reviewed</code>	Timestamp of the last time this card was answered
<code>mastered_at</code>	Timestamp of when the streak hit 3 in the current session
<code>available_at</code>	Exact datetime when this card can reappear after a cooldown

DailyStudyLog

One row per user per calendar day. Used to efficiently compute streaks and dashboard metrics without expensive aggregation queries.

Field	Description
<code>user</code>	FK → User
<code>date</code>	The calendar date
<code>reviewed</code>	Total cards mastered today

Field	Description
<code>created</code>	Total questions approved today
<code>review_streak_met</code>	<code>True</code> if ≥ 50 cards were mastered today
<code>creation_streak_met</code>	<code>True</code> if ≥ 50 questions were created/approved today

3. Question Types

The SRS supports four question types, each with a distinct data schema and UI component.

Multiple Choice Question (MCQ)

- A question stem with **4 or 5 answer options**.
- Exactly one correct answer, identified by `correct_index` (0-based).
- Includes an educational `explanation` field shown after answering.

True / False Cluster

- A **stem** (e.g., "Concerning diabetes mellitus:") followed by a list of `statements`.
- Each statement has two versions: a `true_statement` and a `false_alternative`.
- At quiz time, **up to 4 statements are randomly selected** from the pool. For each, the UI independently and randomly shows either the true or the false version. The student must label each as "T" or "F".
- This means any session could theoretically show all-true or all-false, but statistically each statement has a 50/50 chance of appearing in either form.

Fill in the Gap

- A sentence with 2–5 blanks, marked as `{gap_0}`, `{gap_1}`, etc.
- An **answer bank** is provided with exactly `gap_count × 2` options: half are correct, half are plausible distractors.
- Students drag/select options from the bank into the correct gaps.

Select All That Apply (Applies)

- A question with **multiple correct answers** drawn from a list in the source text.
- Also includes at least 2 plausible distractors that are medically related but incorrect.
- The student must select every correct option without selecting wrong ones.

4. Question Generation Pipeline

When a user highlights text in the Library's Read Mode and selects a question type, the following happens:

```
User selects text → frontend calls POST /api/questions/generate/
```

Step 1: Validation & Rate Limiting

The backend checks:

- All required fields are present (`block_id`, `question_type`, `material_id`, `selected_text`).
- The user does not already have **20 or more pending questions** for that material. If they do, the API returns `HTTP 429` and the user must review existing questions first.

Step 2: Context Building (`context_builder.py`)

This is a critical step that gives the AI model *just enough* context to generate an accurate, medically sound question without being overwhelmed by irrelevant text.

The context builder:

1. Walks **up** the document tree from the selected block to find the **nearest heading** (H1, H2, or H3).
2. Collects that heading and all of its **descendant blocks** in document order.
3. If the nearest heading is an H3, it also prepends the parent H2 title (just the title, not all its content) to give the AI topic scope.
4. Always prepends the H1 heading (the overall document/disease title) for medical accuracy grounding.
5. Serializes all collected blocks with type prefixes (e.g., `[TEXT]`, `[LIST ITEM]`, `[HEADING 2]`).
6. **Caps the total context at 1,000 characters** to keep prompts focused and fast.

The final result is a tightly-scoped string like:

```
[HEADING 1] Malaria
[HEADING 2] Pathophysiology
[HEADING 3] Red Cell Invasion
[TEXT] The merozoite invades the red blood cell by...
```

Step 3: AI Generation (`ai_service.py`)

The backend calls **Google Gemini** (`gemini-3.1-pro-preview`) with:

- A base prompt establishing the medical education context and strict JSON-only output rules.
- Type-specific instructions and schema examples (to reduce hallucination and guide structure).
- A `response_schema` (Pydantic model converted to JSON schema) to enforce structured output.

The AI is instructed to write questions that sound like **standalone medical board questions**, never using phrases like "according to the text."

Step 4: Normalization & Validation (Up to 3 Attempts)

The raw AI response goes through:

1. **JSON parsing** — strips any accidental markdown code fences (`` ```).
2. **Type-specific normalization** — for example, `_normalize_mcq` coerces `correct_index` to an integer and removes extra options; `_normalize_true_false` handles cases where the AI uses alternative key names like `pairs` instead of `statements`.
3. **Pydantic schema validation** — the payload is validated against the strict schema (e.g., `MCQSchema`, `TrueFalseSchema`). If it fails, the process retries up to **3 times** before raising a

`GenerationValidationError`.

Step 5: Creation

If validation passes, a `Question` object is created with `status = 'pending'`. No `QuestionSchedule` is created yet — the question is not in the SRS yet.

5. The Approval & Scheduling Pipeline

A question sits in `pending` status and appears in the **Review Mode** of the Library. Here, the user can:

- **Preview** how the question will look.
- **Edit** the question payload inline (fix wording, correct answers, etc.).
- **Regenerate** the question with a correction instruction sent back to the AI.
- **Approve** the question, entering it into the SRS.
- **Reject / Delete** the question.

What happens on Approval?

When a question's status is changed to `approved` via `PATCH /api/questions/<id>/:`

1. A `QuestionSchedule` is created with:

- `scheduled_date = today + 1 day` (it appears in the queue **tomorrow**, not today, giving students time to create a batch of cards before starting a session).
- `streak = 0`
- `review_count = 0`
- `available_at = null`

2. The `DailyStudyLog` for today is updated:

- `created += 1`
 - `creation_streak_met` is recalculated (True if `created >= 50`).
 - `review_streak_met` is also re-evaluated (True if `reviewed >= 50`, or if the total due count is 0 meaning all cards are mastered).
-

6. The Quiz Session — Loading & Selection

When the user navigates to the Quiz Page, the frontend calls `GET /api/questions/session/`.

Computing `session_total`

The backend computes the total number of cards due for today, which determines the session's "out of X" denominator:

```
cards_mastered_today = QuestionSchedule.objects.filter(mastered_at__date=today)
total_due =
QuestionSchedule.objects.filter(scheduled_date__lte=today).exclude(mastered_at__da
```

```
te=today)
session_total = total_due.count() + cards_mastered_today.count()
```

This means: "how many cards are due today, including the ones already finished."

If `session_total == 0`, the session is over and the user sees the "Session Complete" screen.

Eligibility Filter

Only cards that are **due today or earlier** and **not yet mastered today** are eligible:

```
eligible = QuestionSchedule.objects.filter(
    scheduled_date__lte=today
).exclude(mastered_at__date=today)
```

Excluding Already-Active Cards

The frontend can pass `?exclude_ids=id1,id2,...` to prevent duplicating cards that are already in the local queue. This is used during **backfill** (see Section 8).

7. Quota-Based Card Selection Algorithm

Not all eligible cards are sent to the frontend at once — the session is capped at **30 cards at a time**. These 30 slots are divided into a quota based on the card's "age" in the SRS:

Category	Definition	Quota
New	<code>review_count == 0</code> (never been mastered)	12 cards (40%)
Young	<code>review_count</code> is 1–3 (mastered 1–3 times)	10 cards (35%)
Mature	<code>review_count > 3</code> (mastered 4+ times)	8 cards (25%)

Why this split? New material is given the most exposure because it hasn't been consolidated into long-term memory yet. Mature cards, which the student has proven they know, need less frequent review time.

Shortfall Handling

If any category has fewer cards than its quota (e.g., only 5 new cards exist), the shortfall slots are filled from the remaining un-selected cards of any category, prioritizing cards that are already in progress in the current session (`streak > 0`).

The `has_more` flag

After selecting the 30 primary cards, the backend checks if any eligible cards were left over. If yes, `has_more = True` is returned, which tells the frontend to trigger **backfill** later.

8. The Frontend Queue Runner — Card Ordering

The frontend receives the queue of up to 30 cards and uses a **1-second polling loop** (`setInterval`) to pick the next card to display. This is where the specific order a user sees cards is determined.

Categorizing Cards in the Local Queue

Every card in the `activeQueue` is classified:

- **New/Unattempted:** `streak === 0` AND `availableAt === 0` — never touched in this session.
- **Due Review:** (`streak > 0` OR `availableAt > 0`) AND `availableAt <= now` — previously answered and cooldown has expired.
- **On Cooldown:** `availableAt > now` — answered and waiting to reappear.

Even Alternation (1 New, 1 Review)

When both due reviews and unattempted cards are available simultaneously, the queue runner **alternates** between them using a `lastServedTypeRef`:

```
→ Review card (if last was new)
→ New card (if last was review)
```

This prevents a user from plowing through only new cards and then being bombarded by reviews at the end, or vice versa.

When Only One Type Is Available

- If only due reviews exist, they are served continuously.
- If only new cards exist, they are served continuously.
- If all remaining cards are on cooldown, the UI shows a spinner with a countdown timer showing when the earliest card will be available.

Backfill Trigger

When the `activeQueue` has fewer than **10 unattempted cards**, and `has_more` is true, `tryBackfill()` is called. It fetches the next batch of cards from the server (excluding already-known IDs) and appends them to the local queue seamlessly, without interrupting the current card.

9. Answering a Card — Mastery & Requeue

When the user submits an answer (clicks "Next Question"), the frontend calls:

```
POST /api/questions/session/<question_id>/answer/
Body: { "correct": true | false }
```

Backend Processing (`QuizSessionAnswerView`)

1. **Lazy Streak Reset Check:** Before processing, if `last_reviewed` is from a previous calendar day, the streak is immediately reset to 0 and `mastered_at` is cleared. This handles edge cases where the user left mid-session the previous day.
2. **Update Streak:**
 - If **correct**: `streak += 1`
 - If **incorrect**: `streak = 0`
3. **Check for Mastery:** If `streak >= 3`, the card is **mastered**.

10. The Cooldown (Re-queue) System

If a card is **not** mastered after an answer (streak is between 1–2 for correct, or 0 for incorrect), it gets a **cooldown** before it can reappear:

Result	Cooldown
Correct (but not mastered)	Exactly 5 minutes (<code>available_at = now + 5 minutes</code>)
Incorrect	Exactly 3 minutes (<code>available_at = now + 3 minutes</code>)

Design note: The `srs.py` file defines min/max ranges (`REQUEUE_CORRECT_MIN = 5, MAX = 10`, etc.), but the live `views.py` implementation uses fixed 5-minute and 3-minute delays without randomization. The shorter delay for incorrect cards means you see it sooner when you struggle, helping reinforce the material faster.

The backend returns the `available_at` timestamp in milliseconds to the frontend. The frontend re-inserts the card into the `activeQueue` with the updated `streak` and `availableAt`. The queue runner's 1-second poll will pick it up automatically once the timestamp has passed.

11. The Mastery Streak & Graduation

A card is **mastered** when `streak >= 3` — meaning the user answered it correctly **3 consecutive times in a single session**.

What happens at mastery:

1. `mastered_at` is set to `now`.
2. `review_count` is incremented by 1.
3. A new `scheduled_date` is calculated (see Section 12).
4. `available_at` is cleared to `null` — the card is done for the day.
5. The card is **not** sent back to the frontend queue. Instead, the `completed` counter in the session header increments.
6. The `DailyStudyLog.reviewed` count is updated to the total number of cards mastered today.

The mastery UI on the quiz card shows **3 dots** (● ● ●). Each correct answer fills one dot (○ → ●). An incorrect answer resets all dots to empty.

12. Inter-Session Scheduling — The Next Review Date

Once a card is mastered, `find_next_review_date()` in `srs.py` calculates when it will appear again.

```
def find_next_review_date(user, base_date: date, review_count: int = 0) -> date:
    interval_days = 3 if review_count < 3 else 5
    return base_date + timedelta(days=interval_days)
```

review_count (before this mastery)	Interval	Next Review
0 (first mastery ever)	+3 days	3 days from now
1	+3 days	3 days from now
2	+3 days	3 days from now
3+ (mature card)	+5 days	5 days from now

The interval extends once a card has been mastered more than 3 times, reflecting that deeply learned material needs less frequent review.

Note: The `SLOT_SIZE = 100` constant in `srs.py` defines a cap of 100 cards scheduled per day. This is defined but not yet actively enforced as a scheduling constraint in the current version (cards are scheduled on `base_date + interval` directly without checking slot density).

13. Daily Study Streak System

A **study streak** is a consecutive number of days where the user meets *both* criteria:

1. **Review Streak Met:** Mastered ≥ 50 questions in a single day.
2. **Creation Streak Met:** Approved ≥ 50 new questions in a single day.

Both must be met on the same day for that day to "count" toward the streak. This is called the **unified streak**.

How the Streak is Counted

The backend walks backwards from today, day by day, counting consecutive days where `review_streak_met = True` AND `creation_streak_met = True`:

```
check_date = today
if not (today's log has both streak_met):
    check_date -= 1 day # today doesn't count yet

while DailyStudyLog where date=check_date AND both streak_met:
    current_streak += 1
    check_date -= 1 day
```

If today's goal hasn't been met yet, it looks back from yesterday so the current in-progress day doesn't break an active streak.

How Streaks are Broken

A streak breaks the moment a day passes where either `review_streak_met = False` or `creation_streak_met = False`. There is no grace period.

Longest Streak

The longest streak is computed by scanning all "unified" days in order and finding the longest consecutive run.

14. Dashboard Metrics

The Dashboard (`GET /api/questions/dashboard/stats/`) returns:

Metric	How Computed
<code>reviewed_today</code>	<code>DailyStudyLog.reviewed</code> for today
<code>created_today</code>	<code>DailyStudyLog.created</code> for today
<code>review_streak_minimum</code>	Constant: 50
<code>creation_streak_minimum</code>	Constant: 50
<code>review_streak_met</code>	Whether today's log has <code>review_streak_met = True</code>
<code>creation_streak_met</code>	Whether today's log has <code>creation_streak_met = True</code>
<code>current_review_streak</code>	Current unified streak count
<code>current_creation_streak</code>	(Same as review streak — they are unified)
<code>longest_review_streak</code>	Longest unified streak ever
<code>due_count</code>	Cards due today that haven't been mastered today
<code>cards_mastered_today</code>	Count of <code>QuestionSchedule</code> where <code>mastered_at__date = today</code>
<code>monthly_activity</code>	Array of daily stats for the current month

The `monthly_activity` array drives the calendar heat map on the dashboard, showing which days had both streaks met, only one, or neither.

15. Streak Reset Logic

Card streaks (`QuestionSchedule.streak`) are session-local. They must reset between calendar days to ensure the 3-consecutive-correct requirement applies freshly each day.

Two Reset Mechanisms

1. Eager Reset (in `QuizSessionView.get()`)

When building the quiz session, the backend scans all eligible cards and checks:

- If `mastered_at` is from a **previous day** → the card was mastered yesterday and is due again; reset streak.
- If `last_reviewed` is from a **previous day** AND `streak > 0` → partial progress from a previous session; reset streak.

This ensures cards enter the new session's queue with a clean slate.

2. Lazy Reset (in `QuizSessionAnswerView.post()`)

A safety net: at the moment an answer is submitted, if `last_reviewed` is from a previous day, the streak is reset before applying the new answer. This handles rare cases where a card avoided the eager reset (e.g., via backfill late in the session).

16. End-to-End Flow Walkthrough

Here is the complete lifecycle of a single question:

1. CREATION

User reads material in Library → highlights text → selects question type
→ AI generates question → Question created (status: pending)

2. REVIEW IN LIBRARY (Review Mode)

User sees the pending question → previews, edits if needed → Approves
→ QuestionSchedule created: `scheduled_date = today + 1 day`, `streak = 0`

3. NEXT DAY – QUIZ SESSION STARTS

User opens Quiz Page → GET `/api/questions/session/`
→ Card appears in queue (`review_count = 0`, "new" category)

4. IN-SESSION ROUND 1 – ANSWERED INCORRECTLY

User answers wrong → streak stays 0
→ Card gets 3-minute cooldown (`available_at = now + 3 min`)
→ Card pushed to back of frontend queue

5. IN-SESSION ROUND 2 (3 min later) – ANSWERED CORRECTLY

User answers correct → streak = 1
→ Card gets 5-minute cooldown
→ Card pushed back into queue

6. IN-SESSION ROUND 3 (5 min later) – ANSWERED CORRECTLY

User answers correct → streak = 2
→ Card gets another 5-minute cooldown

7. IN-SESSION ROUND 4 (5 min later) – ANSWERED CORRECTLY

User answers correct → streak = 3 → MASTERED!
→ `mastered_at = now`

- `review_count = 1`
- `scheduled_date = today + 3 days`
- Card removed from active queue
- "Completed" counter increments

8. THREE DAYS LATER – NEXT REVIEW

- User opens Quiz Page → Card enters queue (`review_count = 1`, "young" category)
- User must reach `streak = 3` again from scratch
- On mastery: `scheduled_date = today + 3 days`, `review_count = 2`

9. AFTER 3+ MASTERIES – MATURE CARD

- `review_count` becomes 3 → card is now "mature"
- Interval extends to 5 days between sessions
- Card appears less frequently in quota (8 slots vs 12 for new)

17. Key Constants Reference

All tunable constants in the SRS:

Constant	Location	Value	Meaning
<code>STREAK_TARGET</code>	<code>srs.py</code>	3	Consecutive correct answers to master a card
<code>BASE_INTERVAL_DAYS</code>	<code>srs.py</code>	3	Days until next review for cards with <code>review_count < 3</code>
<code>SLOT_SIZE</code>	<code>srs.py</code> , <code>views.py</code>	100	Defined max cards per day slot (not yet actively enforced)
<code>REQUEUE_CORRECT_MIN/MAX</code>	<code>srs.py</code>	5–10 min	Range for correct-but-not-mastered cooldown (defined but not used in live logic)
<code>REQUEUE_WRONG_MIN/MAX</code>	<code>srs.py</code>	3–5 min	Range for incorrect cooldown (defined but not used in live logic)
Live correct cooldown	<code>views.py</code>	5 minutes	Fixed delay after a correct (non-mastering) answer
Live incorrect cooldown	<code>views.py</code>	3 minutes	Fixed delay after an incorrect answer
Mature interval	<code>srs.py</code>	5 days	Inter-session interval for <code>review_count >= 3</code>
Session batch size	<code>views.py</code>	30 cards	Total cards loaded into a session at once
New card quota	<code>views.py</code>	12 (40%)	New cards per session batch

Constant	Location	Value	Meaning
Young card quota	<code>views.py</code>	10 (35%)	Young cards per session batch
Mature card quota	<code>views.py</code>	8 (25%)	Mature cards per session batch
Backfill trigger	<code>QuizPage.tsx</code>	< 10 unattempted	Trigger fetching more cards from backend
Pending cap	<code>views.py</code>	20 questions	Max pending questions per material before generation is blocked
<code>REVIEW_STREAK_MINIMUM</code>	<code>views.py</code>	50 cards	Cards mastered per day to meet review streak goal
<code>CREATION_STREAK_MINIMUM</code>	<code>views.py</code>	50 questions	Questions approved per day to meet creation streak goal
Max context chars	<code>context_builder.py</code>	1,000 chars	Context sent to AI per generation request
AI max retries	<code>ai_service.py</code>	3 attempts	Generation retries before returning an error

This document was generated from a direct reading of the source code. Refer to the individual source files for the authoritative implementation:

- `backend/quiz/models.py` — Data model
- `backend/quiz/srs.py` — SRS constants and interval logic
- `backend/quiz/views.py` — Session loading, answer processing, dashboard stats
- `backend/quiz/ai_service.py` — Question generation and normalization
- `backend/quiz/context_builder.py` — AI context construction
- `backend/quiz/schemas.py` — Question payload schemas
- `frontend/src/pages/QuizPage.tsx` — Session management, queue runner
- `frontend/src/components/TrueFalseReview.tsx` — T/F presentation logic